Static program analysis

In computer science, **static program analysis** (also known as **static analysis** or **static simulation**) is the analysis of computer programs performed without executing them, in contrast with dynamic program analysis, which is performed on programs during their execution in the integrated environment.[1][2]

The term is usually applied to analysis performed by an automated tool, with human analysis typically being called "program understanding", program comprehension, or code review. In the last of these, software inspection and software walkthroughs are also used. In most cases the analysis is performed on some version of a program's source code, and, in other cases, on some form of its object code.

> Two leading approaches to resource certification have been Static Analysis (SA) and Implicit Computational Complexity (ICC). SA is algorithmic in nature: it focuses on a broad programming language of choice, and seeks to determine by syntactic means whether given programs in that language are feasible. In contrast, ICC attempts to create from the outset specialized programming languages or methods that delineate a complexity class. Thus, SA's focus is on compile time, making no demand on the programmer; whereas ICC is a language-design discipline."
>
> —D. Leivant (2020)[3]

The discipline of static analysis should not be confused with linting, which is the process of checking for coding style mistakes.

Rationale

The sophistication of the analysis performed by tools varies from those that only consider the behaviour of individual statements and declarations,[4] to those that include the complete source code of a program in their analysis. The uses of the information obtained from the analysis vary from highlighting possible coding errors (e.g., the lint tool) to formal methods that mathematically prove properties about a given program (e.g., its behaviour matches that of its specification).

Software metrics and reverse engineering can be described as forms of static analysis. Deriving software metrics and static analysis are increasingly deployed together, especially in creation of embedded systems, by defining so-called *software quality objectives*.[5]

A growing commercial use of static analysis is in the verification of properties of software used in safety-critical computer systems and locating potentially vulnerable code.[6] For example, the following industries have identified the use of static code analysis as a means of improving the quality of increasingly sophisticated and complex software:

1. Medical software: The US Food and Drug Administration (FDA) has identified the use of static analysis for medical devices.[7]
2. Nuclear software: In the UK the Office for Nuclear Regulation (ONR) recommends the use of static analysis on reactor protection systems.[8]
3. Aviation software (in combination with dynamic analysis).[9]
4. Automotive & Machines (functional safety features form an integral part of each automotive product development phase, ISO 26262, section 8).

A study in 2012 by VDC Research reported that 28.7% of the embedded software engineers surveyed use static analysis tools and 39.7% expect to use them within 2 years.[10] A study from 2010 found that 60% of the interviewed developers in European research projects made at least use of their basic IDE built-in static analyzers. However, only about 10% employed an additional other (and perhaps more advanced) analysis tool.[11]

In the application security industry the name static application security testing (SAST) is also used. SAST is an important part of Security Development Lifecycles (SDLs) such as the SDL defined by Microsoft[12] and a common practice in software companies.[13]

Tool types

The OMG (Object Management Group) published a study regarding the types of software analysis required for software quality measurement and assessment. This document on "How to Deliver Resilient, Secure, Efficient, and Easily Changed IT Systems in Line with CISQ Recommendations" describes three levels of software analysis.[14]

Unit Level
: Analysis that takes place within a specific program or subroutine, without connecting to the context of that program.

Technology Level
: Analysis that takes into account interactions between unit programs to get a more holistic and semantic view of the overall program in order to find issues and avoid obvious false positives.

System Level
: Analysis that takes into account the interactions between unit programs, but without being limited to one specific technology or programming language.

A further level of software analysis can be defined.

Mission/Business Level
: Analysis that takes into account the business/mission layer terms, rules and processes that are implemented within the software system for its operation as part of enterprise or program/mission layer activities. These elements are implemented without being limited to one specific technology or programming language and in many cases are distributed across multiple languages, but are statically extracted and analyzed for system understanding for mission assurance.

Formal methods

Formal methods is the term applied to the analysis of software (and computer hardware) whose results are obtained purely through the use of rigorous mathematical methods. The mathematical techniques used include denotational semantics, axiomatic semantics, operational semantics, and abstract interpretation.

By a straightforward reduction to the halting problem, it is possible to prove that (for any Turing complete language), finding all possible run-time errors in an arbitrary program (or more generally any kind of violation of a specification on the final result of a program) is undecidable: there is no mechanical method that can always answer truthfully whether an arbitrary program may or may not exhibit runtime errors. This result dates from the works of Church, Gödel and Turing in the 1930s (see: Halting problem and Rice's theorem). As with many undecidable questions, one can still attempt to give useful approximate solutions.

Some of the implementation techniques of formal static analysis include:[15]

- Abstract interpretation, to model the effect that every statement has on the state of an abstract machine (i.e., it 'executes' the software based on the mathematical properties of each statement and declaration). This abstract machine over-approximates the behaviours of the system: the abstract system is thus made simpler to analyze, at the expense of *incompleteness* (not every property true of the original system is true of the abstract system). If properly done, though, abstract interpretation is *sound* (every property true of the abstract system can be mapped to a true property of the original system).[16]
- Data-flow analysis, a lattice-based technique for gathering information about the possible set of values;
- Hoare logic, a formal system with a set of logical rules for reasoning rigorously about the correctness of computer programs. There is tool support for some programming languages (e.g., the SPARK programming language (a subset of Ada) and the Java Modeling Language—JML—using ESC/Java and ESC/Java2, Frama-C WP (weakest precondition) plugin for the C language extended with ACSL (ANSI/ISO C Specification Language)).
- Model checking, considers systems that have finite state or may be reduced to finite state by abstraction;
- Symbolic execution, as used to derive mathematical expressions representing the value of mutated variables at particular points in the code.
- Nullable reference analysis

Data-driven static analysis

Data-driven static analysis leverages extensive codebases to infer coding rules and improve the accuracy of the analysis.[17][18] For instance, one can use all Java open-source packages available on GitHub to learn good analysis strategies. The rule inference can use machine learning techniques.[19] It is also possible to learn from a large amount of past fixes and warnings.[17]

Remediation

Static analyzers produce warnings. For certain types of warnings, it is possible to design and implement automated remediation techniques. For example, Logozzo and Ball have proposed automated remediations for C# *cccheck*.[20]

See also

- Code audit
- Documentation generator
- Formal verification
- FX-87
- Implicit computational complexity
- ISO 26262
- ISO 9126 (now ISO 25000 series)
- Lint (software)
- List of tools for static code analysis
- Semantics (computer science)
- Shape analysis
- Software quality
- Software quality assurance
- SonarQube

References

1. Wichmann, B. A.; Canning, A. A.; Clutterbuck, D. L.; Winsbarrow, L. A.; Ward, N. J.; Marsh, D. W. R. (Mar 1995). "Industrial Perspective on Static Analysis" (https://web.archive.org/web/20110927010304/http://www.ida.liu.se/~TDDC90/papers/industrial95.pdf) (PDF). *Software Engineering Journal*. **10** (2): 69–75. doi:10.1049/sej.1995.0010 (https://doi.org/10.1049%2Fsej.1995.0010). Archived from the original (http://www.ida.liu.se/~TDDC90/papers/industrial95.pdf) (PDF) on 2011-09-27.
2. Egele, Manuel; Scholte, Theodoor; Kirda, Engin; Kruegel, Christopher (2008-03-05). "A survey on automated dynamic malware-analysis techniques and tools" (https://doi.org/10.1145/2089125.2089126). *ACM Computing Surveys*. **44** (2): 6:1–6:42. doi:10.1145/2089125.2089126 (https://doi.org/10.1145%2F2089125.2089126). ISSN 0360-0300 (https://search.worldcat.org/issn/0360-0300). S2CID 1863333 (https://api.semanticscholar.org/CorpusID:1863333).
3. Leivant, Daniel (2020). "A Generic Imperative Language for Polynomial Time". arXiv:1911.04026 (https://arxiv.org/abs/1911.04026) [cs.CC (https://arxiv.org/archive/cs.CC)].
4. Khatiwada, Saket; Tushev, Miroslav; Mahmoud, Anas (2018-01-01). "Just enough semantics: An information theoretic approach for IR-based software bug localization" (https://linkinghub.elsevier.com/retrieve/pii/S0950584916302269). *Information and Software Technology*. **93**: 45–57. doi:10.1016/j.infsof.2017.08.012 (https://doi.org/10.1016%2Fj.infsof.2017.08.012).
5. "Software Quality Objectives for Source Code" (http://web1.see.asso.fr/erts2010/Site/0ANDGY78/Fichier/PAPIERS%20ERTS%202010/ERTS2010_0035_final.pdf) Archived (https://web.archive.org/web/20150604203133/http://web1.see.asso.fr/erts2010/Site/0ANDGY78/Fichier/PAPIERS%20ERTS%202010/ERTS2010_0035_final.pdf) 2015-06-04 at the Wayback Machine (PDF). *Proceedings: Embedded Real Time Software and Systems 2010 Conference*, ERTS2010.org, Toulouse, France: Patrick Briand, Martin Brochet, Thierry Cambois, Emmanuel Coutenceau, Olivier Guetta, Daniel Mainberte, Frederic Mondot, Patrick Munier, Loic Noury, Philippe Spozio, Frederic Retailleau.
6. *Improving Software Security with Precise Static and Runtime Analysis* (http://research.microsoft.com/en-us/um/people/livshits/papers/pdf/thesis.pdf) Archived (https://web.archive.org/web/20110605125310/http://research.microsoft.com/en-us/um/people/livshits/papers/pdf/thesis.pdf) 2011-06-05 at the Wayback Machine (PDF), Benjamin Livshits, section 7.3 "Static Techniques for Security". Stanford doctoral thesis, 2006.
7. FDA (2010-09-08). "Infusion Pump Software Safety Research at FDA" (https://web.archive.org/web/20100901084658/https://www.fda.gov/MedicalDevices/ProductsandMedicalProcedures/GeneralHospitalDevicesandSupplies/InfusionPumps/ucm202511.htm). Food and Drug Administration. Archived from the original (https://www.fda.gov/MedicalDevices/ProductsandMedicalProcedures/GeneralHospitalDevicesandSupplies/InfusionPumps/ucm202511.htm) on 2010-09-01. Retrieved 2010-09-09.

8. Computer based safety systems - technical guidance for assessing software aspects of digital computer based protection systems, "Computer based safety systems" (http://webarchive.nationalarchives.gov.uk/20130104193206/http://www.hse.gov.uk/nuclear/operational/tech_asst_guides/tast046.pdf) (PDF). Archived from the original (http://www.hse.gov.uk/nuclear/operational/tech_asst_guides/tast046.pdf) (PDF) on January 4, 2013. Retrieved May 15, 2013.
9. Position Paper CAST-9. Considerations for Evaluating Safety Engineering Approaches to Software Assurance (http://www.faa.gov/aircraft/air_cert/design_approvals/air_software/cast/cast_papers/media/cast-9.pdf) Archived (https://web.archive.org/web/20131006134233/http://www.faa.gov/aircraft/air_cert/design_approvals/air_software/cast/cast_papers/media/cast-9.pdf) 2013-10-06 at the Wayback Machine // FAA, Certification Authorities Software Team (CAST), January, 2002: "Verification. A combination of both static and dynamic analyses should be specified by the applicant/developer and applied to the software."
10. VDC Research (2012-02-01). "Automated Defect Prevention for Embedded Software Quality" (http://alm.parasoft.com/embedded-software-vdc-report/). VDC Research. Archived (https://web.archive.org/web/20120411211422/http://alm.parasoft.com/embedded-software-vdc-report/) from the original on 2012-04-11. Retrieved 2012-04-10.
11. Prause, Christian R., René Reiners, and Silviya Dencheva. "Empirical study of tool support in highly distributed research projects." Global Software Engineering (ICGSE), 2010 5th IEEE International Conference on. IEEE, 2010 https://ieeexplore.ieee.org/Xplore/login.jsp?url=%2Fielx5%2F5581168%2F5581493%2F05581551.pdf&authDecision=-203
12. M. Howard and S. Lipner. The Security Development Lifecycle: SDL: A Process for Developing Demonstrably More Secure Software. Microsoft Press, 2006. ISBN 978-0735622142
13. Achim D. Brucker and Uwe Sodan. Deploying Static Application Security Testing on a Large Scale (https://www.brucker.ch/bibliography/download/2014/brucker.ea-sast-expierences-2014.pdf) Archived (https://web.archive.org/web/20141021065105/http://www.brucker.ch/bibliography/download/2014/brucker.ea-sast-expierences-2014.pdf) 2014-10-21 at the Wayback Machine. In GI Sicherheit 2014. Lecture Notes in Informatics, 228, pages 91-101, GI, 2014.
14. "OMG Whitepaper | CISQ - Consortium for Information & Software Quality" (http://www.omg.org/CISQ_compliant_IT_Systemsv.4-3.pdf) (PDF). Archived (https://web.archive.org/web/20131228132152/http://www.omg.org/CISQ_compliant_IT_Systemsv.4-3.pdf) (PDF) from the original on 2013-12-28. Retrieved 2013-10-18.
15. Vijay D'Silva; et al. (2008). "A Survey of Automated Techniques for Formal Software Verification" (http://www.kroening.com/papers/tcad-sw-2008.pdf) (PDF). Transactions On CAD. Archived (https://web.archive.org/web/20160304074248/http://www.kroening.com/papers/tcad-sw-2008.pdf) (PDF) from the original on 2016-03-04. Retrieved 2015-05-11.
16. Jones, Paul (2010-02-09). "A Formal Methods-based verification approach to medical device software analysis" (https://web.archive.org/web/20110710185427/http://embeddeddsp.embedded.com/design/opensource/222700533). Embedded Systems Design. Archived from the original (http://embeddeddsp.embedded.com/design/opensource/222700533) on July 10, 2011. Retrieved 2010-09-09.
17. "Learning from other's mistakes: Data-driven code analysis" (https://www.slideshare.net/japh44/talk-handout-46938511). *www.slideshare.net*. 13 April 2015.
18. Söderberg, Emma; Church, Luke; Höst, Martin (2021-06-21). "Open Data-driven Usability Improvements of Static Code Analysis and its Challenges" (https://doi.org/10.1145/3463274.3463808). *Evaluation and Assessment in Software Engineering*. EASE '21. New York, NY, USA: Association for Computing Machinery. pp. 272–277. doi:10.1145/3463274.3463808 (https://doi.org/10.1145%2F3463274.3463808). ISBN 978-1-4503-9053-8.
19. Oh, Hakjoo; Yang, Hongseok; Yi, Kwangkeun (2015). "Learning a strategy for adapting a program analysis via bayesian optimisation". *Proceedings of the 2015 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications - OOPSLA 2015* (https://ora.ox.ac.uk/objects/uuid:f656bcfd-ec1b-477c-9185-ff2c7490a207). pp. 572–588. doi:10.1145/2814270.2814309 (https://doi.org/10.1145%2F2814270.2814309). ISBN 9781450336895. S2CID 13940725 (https://api.semanticscholar.org/CorpusID:13940725).
20. Logozzo, Francesco; Ball, Thomas (2012-11-15). "Modular and verified automatic program repair" (https://dx.doi.org/10.1145/2398857.2384626). *ACM SIGPLAN Notices*. **47** (10): 133–146. doi:10.1145/2398857.2384626 (https://doi.org/10.1145%2F2398857.2384626). ISSN 0362-1340 (https://search.worldcat.org/issn/0362-1340).

Further reading

- Ayewah, Nathaniel; Hovemeyer, David; Morgenthaler, J. David; Penix, John; Pugh, William (2008). "Using Static Analysis to Find Bugs". *IEEE Software*. **25** (5): 22–29. CiteSeerX 10.1.1.187.8985 (https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.187.8985). doi:10.1109/MS.2008.130 (https://doi.org/10.1109%2FMS.2008.130). S2CID 20646690 (https://api.semanticscholar.org/CorpusID:20646690).
- Brian Chess, Jacob West (Fortify Software) (2007). *Secure Programming with Static Analysis*. Addison-Wesley. ISBN 978-0-321-42477-8.
- Flemming Nielson; Hanne R. Nielson; Chris Hankin (2004-12-10). *Principles of Program Analysis* (1999 (corrected 2004) ed.). Springer. ISBN 978-3-540-65410-0.
- "Abstract interpretation and static analysis," (http://santos.cis.ksu.edu/schmidt/Escuela03/home.html) International Winter School on Semantics and Applications 2003, by David A. Schmidt (http://people.cis.ksu.edu/~schmidt/)

Retrieved from "https://en.wikipedia.org/w/index.php?title=Static_program_analysis&oldid=1344927565"